

41811海南部署

基本操作

进10.114.255.26

user

000000

代码块

```
1 source ~/.bashrc
2
3 Path:
4 /home/user/医保
5
6 环境:
7 conda activate my_arm_env
8
9
```

部署

前后端宿主机端口需开于8000-8080中间

代码块

```
1 10.114.255.26
2 ssh
3 后端
4 cd /home/user/医保/bushu/41811_docker_backend
5 sudo docker-compose down #关服务
6 sudo docker-compose up
7
8
9 前端
10 cd /home/user/医保/bushu/41811_deploy
11 dist (npm run build)
12 base_url='10.114.255.26:8045/api'
13 unzip dist.zip
14 sudo docker-compose down
15 sudo docker-compose up -d --build
16
```

```
17
18 rdp
19 user:administrator
20 pass:Ecust@1234
21 10.114.255.25:8045
```

数据库部署

db.sqlite3

建立data文件夹在后端目录下，把dbsqlite3传进去。

mysql

所需（放于同一目录）

- ☐ medical_insurance.sql
- ☐ docker-compose.yml

代码块

```
1  version: '3.3'
2
3  services:
4    medical_mysql:
5      image: mysql:latest
6      container_name: medical_mysql
7      restart: always
8      environment:
9        MYSQL_ROOT_PASSWORD: hainan
10       MYSQL_DATABASE: medical_41811
11       MYSQL_USER: admin
12       MYSQL_PASSWORD: yibao1234
13       TZ: Asia/Shanghai # 强烈建议加上时区，否则数据库时间会慢8小时
14       # 修改 command，去掉 bind-address（默认就是全开的），改为指定字符集
15       command:
16         --character-set-server=utf8mb4
17         --collation-server=utf8mb4_unicode_ci
18       ports:
19         - "43311:3306"
20       volumes:
21         - ./mysql_data:/var/lib/mysql
22       # 【核心】：让数据库加入前端创建的网络
23       # MySQL 容器第一次启动时，会自动扫描这个特殊的文件夹，并执行里面的 sql 文件
24         - ./hainan_41811.sql:/docker-entrypoint-initdb.d/hainan_41811.sql
25     networks:
```

```
26     default:
27     external:
28     name: 41811_deploy_default # 填入你实际的前端网络名
```

启动 (在screen中运行)

代码块

```
1  docker-compose build
2  docker-compose up
```

可能遇到的问题

代码块

```
1  sudo docker stop <id>
2  sudo docker rm <id>
3  #docker容器名称被占用时
4  docker exec -it yibao_database mysql -uadmin -pyibao1234
5  #进入容器
6
```

Tidb

后端部署（和数据库放在一起了，后续需要替换并分开部署）

所需

☐ 一个包含所有包的镜像

镜像制作

实验室服务器下准备 **dockerfile** + **requirements.txt**

代码块

```
1  FROM python:3.11
2
3  ENV TZ=Asia/Shanghai
4  RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && echo $TZ > /etc/timezone
5
6  WORKDIR /app
7
8  RUN apt-get update && apt-get install -y \
```

```
9     default-libmysqlclient-dev \  
10     && rm -rf /var/lib/apt/lists/*  
11  
12 COPY requirements.txt ./  
13 RUN pip install --no-cache-dir -r requirements.txt -i  
    https://pypi.tuna.tsinghua.edu.cn/simple  
14  
15 COPY . .  
16  
17 # 仅仅声明端口即可，不需要写死 CMD 启动命令  
18 EXPOSE 43320  
19 EXPOSE 8000
```

代码块

```
1 #进到对应路径下  
2 # build镜像  
3 sudo docker build -t backend_41811:latest .  
4 #打包镜像  
5 sudo docker save backend_41811:latest -o /home/ubuntu/backend_41811.tar
```

将获取到的tar包上传至服务器后

代码块

```
1 docker load -i backend_41811.tar
```

☐ 后端代码

config.py修改

代码块

```
1 """路径与运行参数（LLM、数据库、SSH 隧道等在此修改）。"""  
2 import os  
3 from pathlib import Path  
4 from typing import Optional  
5  
6 ROOT = Path(__file__).resolve().parent  
7  
8 TABLES_JSON = ROOT / "tables.json"  
9 INDICATORS_JSON = ROOT / "指标.json"  
10 PROMPT_LOG_JSON = ROOT / "prompt_log.json"  
11 EXTRACT_TABLES_SCRIPT = ROOT / "extract_tables.py"  
12 LOG_DIR = ROOT / "logs"
```

```
13  STATIC_DIR = ROOT / "static"
14
15  PREVIEW_ROW_LIMIT = 200
16  LOG_PREVIEW_ROW_LIMIT = 10
17  SQL_RETRY_MAX = 3
18  # 分子分母比值型一次返回两条 SQL，MySQL 校验更难通过；单独提高重试上限（固定次数，非死循环）
19  SQL_RETRY_MAX_DUAL = 10
20
21  SQL_CACHE_ENABLED = True
22  SQL_CACHE_MAX_ENTRIES = 64
23
24  PROMPT_MAX_COLUMNS_PER_TABLE: Optional[int] = 500
25
26  # — OpenAI 兼容 API —
27  # 修改为本地大模型配置
28  OPENAI_API_KEY = "EMPTY"
29  OPENAI_BASE_URL = "http://10.114.255.26:9233/v1"
30  OPENAI_MODEL = "qwen"
31  # 采样温度；0 有利于相同输入下输出稳定（仍非所有网关/后端 100% 比特级可复现）
32  OPENAI_TEMPERATURE = 0.0
33  # 若网关支持 Chat Completions 的 seed，可设固定整数以增强可复现性；不支持时保持 None，避免多余参数
34  OPENAI_SEED: Optional[int] = None
35  OPENAI_TIMEOUT_SEC = 120.0
36  OPENAI_HTTP_TRUST_ENV = False
37
38  # — SSH 隧道（自动建立，无需手动开终端） —
39  SSH_TUNNEL_ENABLED = False # 设为 False 则直连 MySQL，不走隧道
40  SSH_HOST = "172.20.137.65" # 跳板机 / 服务器 IP
41  SSH_PORT = 22133 # SSH 端口
42  SSH_USER = "ubuntu" # SSH 用户名
43  # 认证二选一：填密码则不用私钥；私钥非空则同时可传（通常只配置一种）
44  # 生产环境建议 export TEXT2SQL_SSH_PASSWORD=...，不在仓库里写明文
45  SSH_PASSWORD = os.environ.get("TEXT2SQL_SSH_PASSWORD", "Tiankong1234")
46  SSH_KEY_PATH = "" # 私钥路径（支持 ~ 展开）；留空则仅用密码
47  SSH_REMOTE_BIND = ("127.0.0.1", 3306) # 服务器上 MySQL 监听的地址和端口
48
49  # — MySQL —
50  # 修改为远程真实的医疗数据库
51  MYSQL_HOST = "10.114.96.57"
52  MYSQL_PORT = 12881
53  MYSQL_USER = "hua.li@tenant_xwjg"
54  MYSQL_PASSWORD = "H@3.14lia"
55  MYSQL_DATABASE = "hua.li"
56  MYSQL_CONNECT_TIMEOUT_SEC = 10
```

☐ docker-compose.yml

代码块

```

1  version: '3.3'
2
3  services:
4    # 1. FastAPI 主后端服务 (跑在 43320)
5    fastapi_service:
6      # 【核心修改】去掉 build, 直接使用刚才 load 进来的离线镜像
7      image: backend_41811:latest
8      container_name: medical_fastapi
9      restart: always
10     command: uvicorn main:app --host 0.0.0.0 --port 43320 --app-dir .
11     environment:
12       - DATABASE_URL=sqlite:///app/data/db.sqlite3
13       - TEXT2SQL_BACKEND_URL=http://medical_text2sql:8000
14     volumes:
15       - ./data:/app/data
16
17     # 2. Text2SQL 大模型翻译服务 (跑在 8000)
18     text2sql_service:
19       # 【核心修改】同上, 共用同一个基础镜像
20       image: backend_41811:latest
21       container_name: medical_text2sql
22       restart: always
23       working_dir: /app/Hainan_SQL-main/text2sql_app
24       command: python run.py
25       volumes:
26         - ./data:/app/data
27
28     # 让它们俩一起加入前端的安全网络
29     networks:
30       default:
31         external:
32           name: 41811_deploy_default

```

前端部署

所需

- ☐ nginx镜像
- ☐ 前端打包dist文件（前端 (.env)需将baseurl(api.ts里已经写好url)）

```
my-vue-app > ⚙️ .env
```

```
1 VITE_API_BASE_URL=http://10.114.255.26:8045
```

☐ dockerfile

代码块

```
1 FROM hub.geekery.cn/nginx:alpine
2
3 # 复制自定义的 nginx 配置
4 COPY default.conf /etc/nginx/conf.d/default.conf
5
6 # 复制打包好的 dist 文件夹
7 COPY dist /usr/share/nginx/html
8
9 EXPOSE 80
10
11 # 设置权限
12 RUN chmod -R 755 /usr/share/nginx/html && \
13     chown -R nginx:nginx /usr/share/nginx/html
14
15 CMD ["nginx", "-g", "daemon off;"]
```

☐ docker-compose.yml

代码块

```
1 version: '3.3'
2
3 services:
4   frontend_nginx:
5     build: ./frontend
6     container_name: medical_frontend
7     restart: always
8     ports:
9       - "8045:80"
10    volumes:
11      # 【关键修改】将外部的 dist 文件夹直接挂载到容器内部
12      - ./dist:/usr/share/nginx/html
13    networks:
14      default:
15        ipam:
16          driver: default
17          config:
```

☐ default.conf

代码块

```
1  server {
2      listen 80;
3      server_name localhost;
4
5      root /usr/share/nginx/html;
6      index index.html;
7
8      # 1. 前端静态文件处理 (支持 Vue 路由的 history 模式)
9      location / {
10         try_files $uri $uri/ /index.html;
11     }
12
13     # 2. 转发 FastAPI 后端请求
14     location /api/ {
15         resolver 127.0.0.11 valid=30s; # Docker 内置 DNS
16         # 【核心修改1】将 fastapi_service 改为真实的容器名: medical_fastapi
17         set $backend http://medical_fastapi:43320;
18         proxy_pass $backend;
19
20         proxy_set_header Host $http_host;
21         proxy_set_header X-Real-IP $remote_addr;
22         proxy_read_timeout 300s;
23     }
24
25     # 3. 转发 Text2SQL 后端请求
26     location /text2sql/ {
27         resolver 127.0.0.11 valid=30s;
28         # 【核心修改2】将 text2sql_service 改为真实的容器名: medical_text2sql
29         set $t2s http://medical_text2sql:8000;
30         proxy_pass $t2s/;
31
32         proxy_set_header Host $host;
33         proxy_set_header X-Real-IP $remote_addr;
34         proxy_read_timeout 300s;
35     }
36 }
```

启动


```
1 docker-compose up -d --build
```

前端更新代码：

代码块

- 1 在前台文件夹下，运行npm run build得到dist文件夹
- 2 将dist文件压缩为zip上传服务器
- 3

启动

代码块

- 1 docker-compose up -d --build

启动

代码块

- 1 docker-compose up -d --build

后端更新代码：

1.更新后台代码（对应地方替换文件）

2.进入后端目录重启docker

```
sudo docker restart medical_fastapi medical_text2sql
```

做下边操作

422 错误修复

根因：`/api/core18/execute/` 端点期望 `indicator_id` 作为 URL 查询参数（`?indicator_id=1`），但前端把 `indicator_id` 放在了 POST 请求体里。

修改 `backend_fastapi/app/routers/core18.py` 第 84-85 行：

改这一行

```
@router.post("/execute/")
```

```
def core18_execute(indicator_id: int, db: Session = Depends(get_db)):
```

为（加上 *body* 参数支持）

```
from fastapi import Body
```

```
@router.post("/execute/")
```

```
def core18_execute(indicator_id: int = Body(None), db: Session = Depends(get_db)):
```

```
    if indicator_id is None:
```

```
        raise HTTPException(status_code=400, detail="indicator_id 不能为空")
```

医院列表为空 + 查询超时修复

根因： `DIM_MDC_ORG` 数据量大，查全量医院本身就慢，加上没有超时限制。

修改 `backend_fastapi/app/routers/system.py` 里 `_query_remote_hospitals()` 函数的 SQL，加 `LIMIT`：

把这条 SQL

```
cur.execute("SELECT MDC_ORG_CD, MDC_ORG_NM FROM DIM_MDC_ORG ORDER BY  
MDC_ORG_NM")
```

改成（只查200条，足够前端用）

```
cur.execute("SELECT MDC_ORG_CD, MDC_ORG_NM FROM DIM_MDC_ORG ORDER BY  
MDC_ORG_NM LIMIT 200")
```

同时把 `read_timeout` 从 30 改成 10 秒，防止长时间卡死：

```
read_timeout=10, # 原来是 30
```

`indicators.py` 里的 `fallback` 函数也做同样的两处修改。

这两个改完之后，422 和医院列表的问题应该都能解决。改完记得重启后端服务。